

บทที่ 6

คำสั่งวนซ้ำ

Iterative statements

อาจารย์ผู้สอน:

- ▶ อ. ดร.พงษ์ศร จันทรียอย
วิทยาลัยการคอมพิวเตอร์ มหาวิทยาลัยขอนแก่น

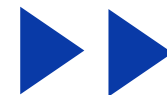


วัตถุประสงค์การเรียนรู้

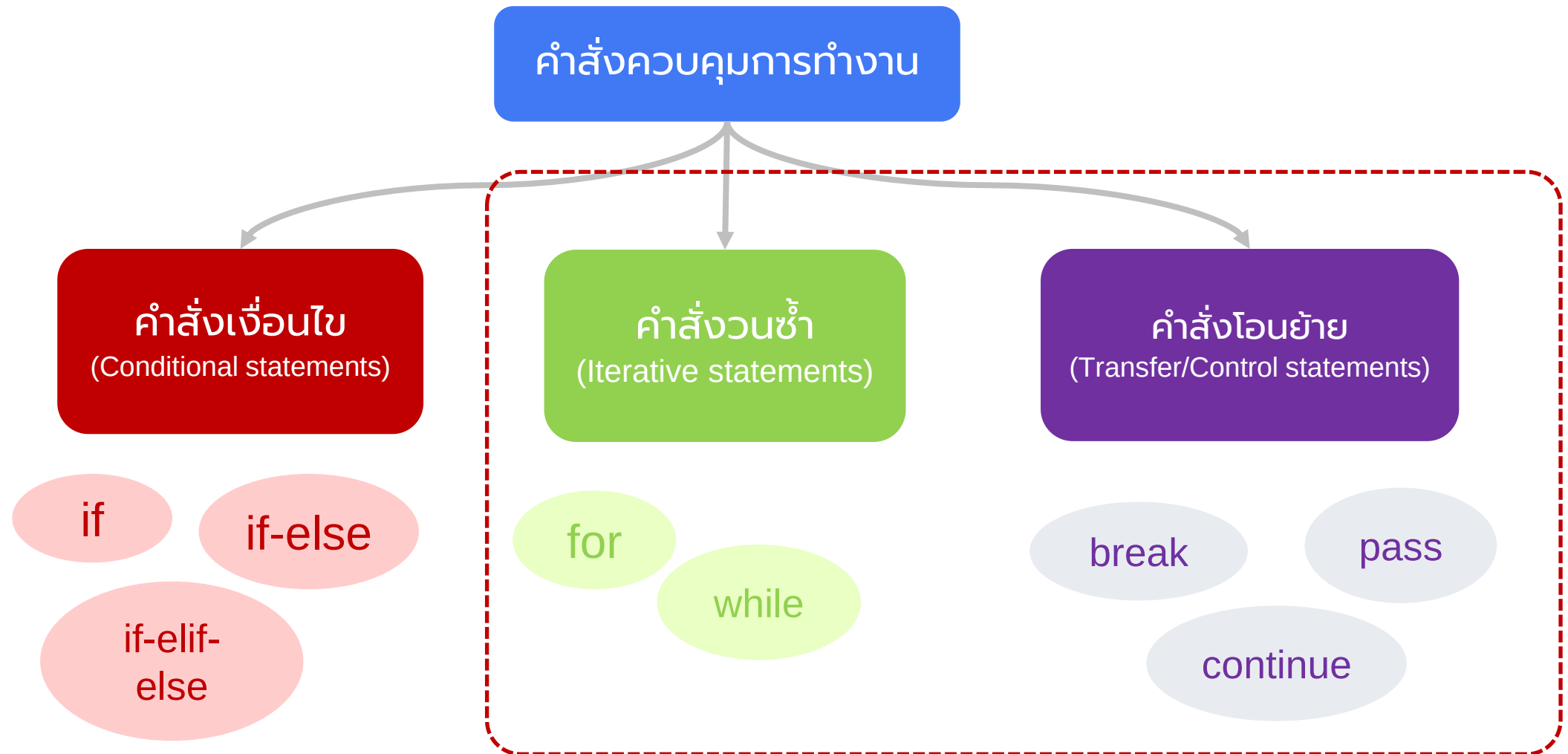
- เพื่อให้ผู้เรียนสามารถอธิบายแนวคิดและความสำคัญของคำสั่งวนซ้ำในการเขียนโปรแกรม
- เพื่อให้ผู้เรียนเข้าใจและใช้งานคำสั่งวนซ้ำได้อย่างถูกต้องและเหมาะสมกับปัญหาที่กำหนดได้อย่างมีประสิทธิภาพ

เนื้อหา

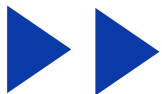
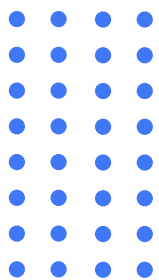
01. คำสั่งวนซ้ำ while และ for
02. คำสั่งโอนย้าย (break, continue, pass)
03. การวนซ้ำแบบซ้อนกัน (Nested loop)



คำสั่งควบคุมการทำงาน (Control Flow Statements)



คำสั่งวนซ้ำ while



ตัวอย่างการวนซ้ำ



ตัวอย่างการวนซ้ำ

Login Window

Entry password invalid, Try again!

Username

Password

Login

Sign in

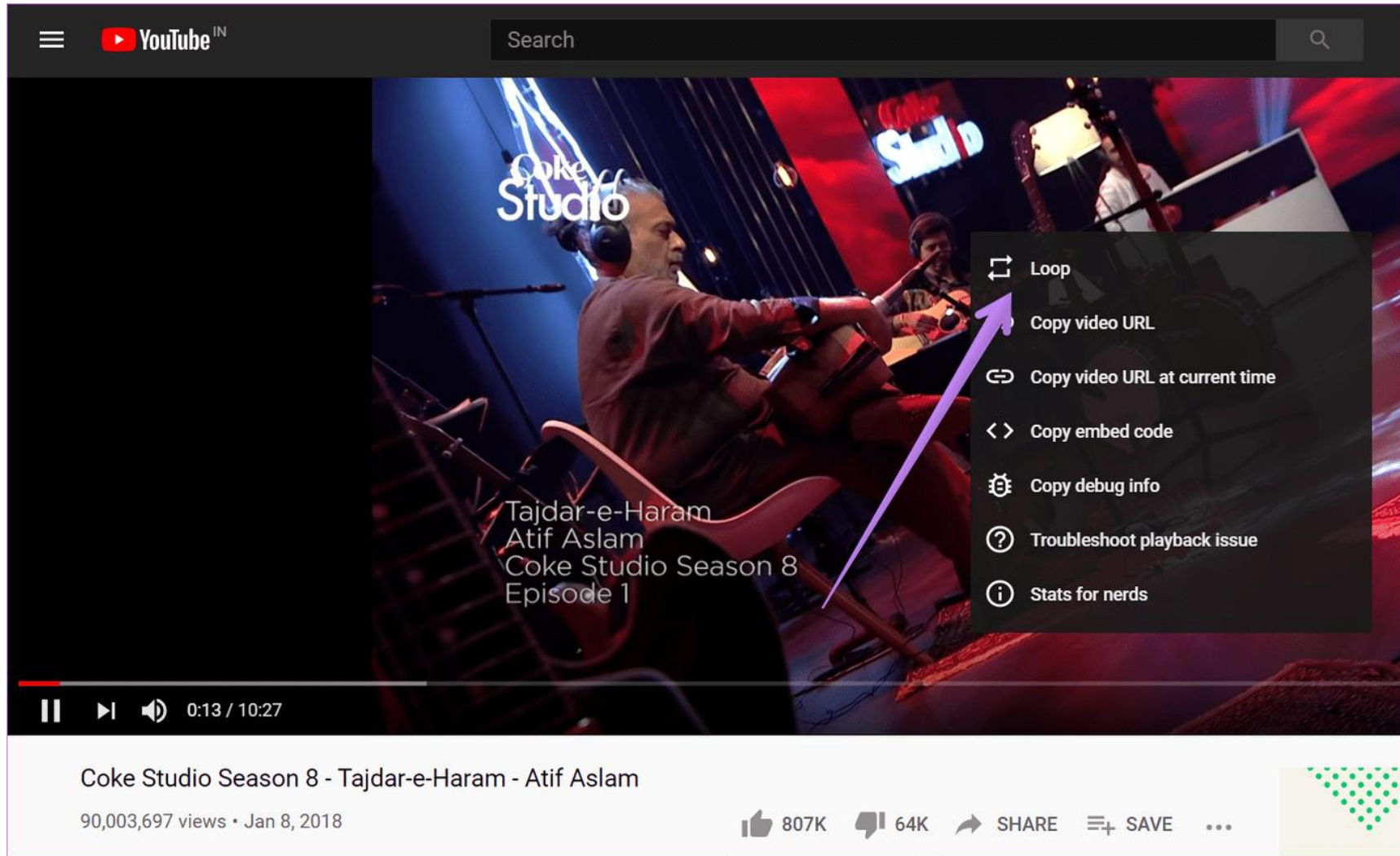
! Invalid username or password

User name

Password

Sign in

ตัวอย่างการวนซ้ำ



The screenshot shows a YouTube video player interface. The video is titled "Coke Studio Season 8 - Tajdar-e-Haram - Atif Aslam". The video player shows a man playing a guitar on a stage. A context menu is open over the video, displaying several options. A purple arrow points to the "Loop" option, which is the first item in the menu. The other options are "Copy video URL", "Copy video URL at current time", "Copy embed code", "Copy debug info", "Troubleshoot playback issue", and "Stats for nerds".

YouTube

Search

Coke Studio

Tajdar-e-Haram
Atif Aslam
Coke Studio Season 8
Episode 1

0:13 / 10:27

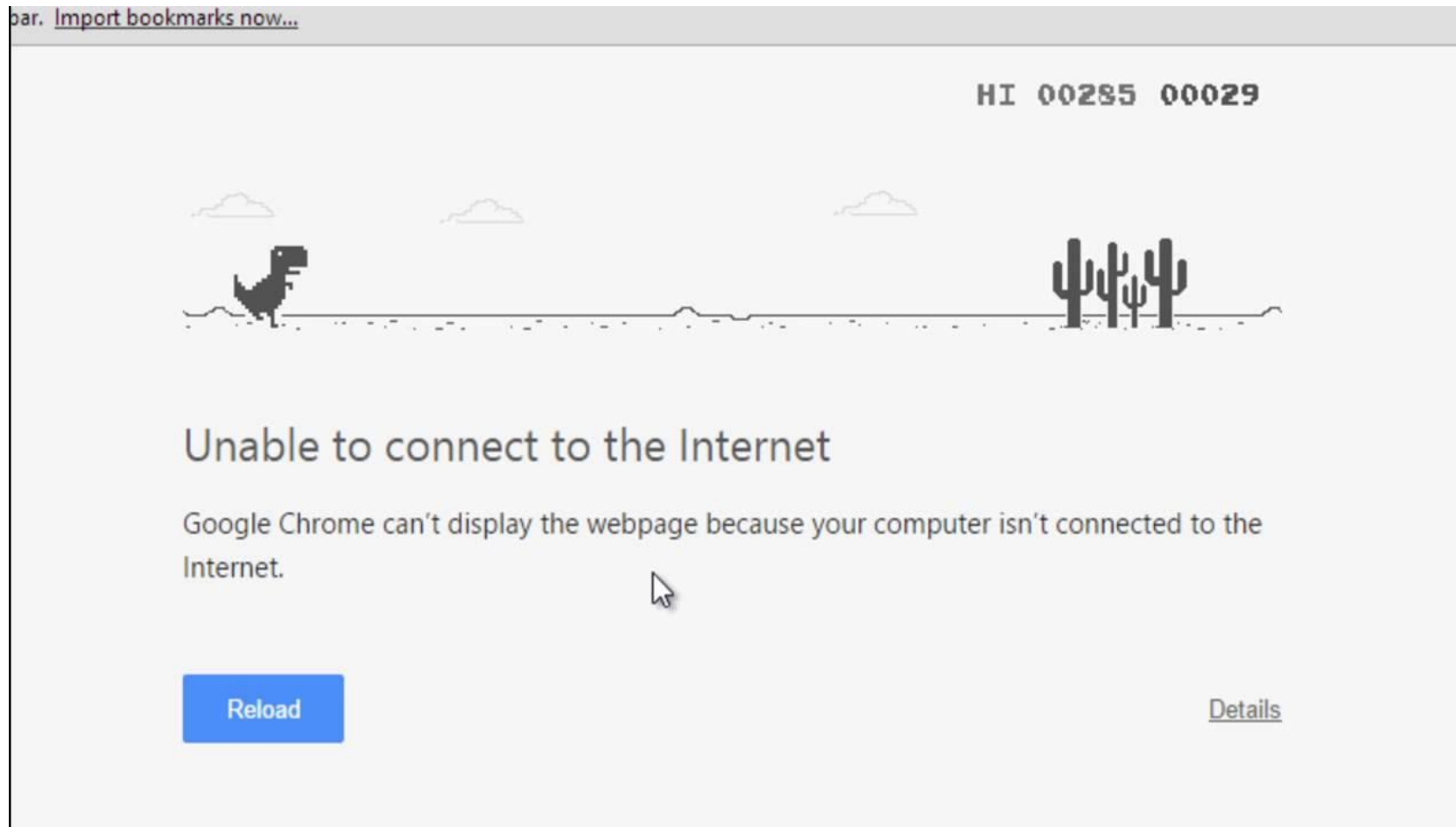
Coke Studio Season 8 - Tajdar-e-Haram - Atif Aslam

90,003,697 views • Jan 8, 2018

807K 64K SHARE SAVE ...

- Loop
- Copy video URL
- Copy video URL at current time
- Copy embed code
- Copy debug info
- Troubleshoot playback issue
- Stats for nerds

ตัวอย่างการวนซ้ำ



ตัวอย่างการวนซ้ำ



ตัวอย่างการวนซ้ำ



ตัวอย่างการวนซ้ำ

- ซีมอไซค์รอบวงเวียนสัก 3 รอบ



ตัวอย่างโปรแกรมที่ไม่มีการวนซ้ำและมีการวนซ้ำ

- **จงแสดงตัวเลขตั้งแต่ 1 - 100**

```
print(1)
print(2)
print(3)
print(4)
print(5)
#เขียนไปเรื่อยๆ จนถึง 100
print(100)
```

```
print(1, 2, 3, 4, 5,
      'เขียนไปเรื่อยๆ',
      100, sep='\\n')
```

```
for i in range(1, 101):
    print(i)
```

คำสั่งวนซ้ำ: while

- คำสั่ง while

Syntax:

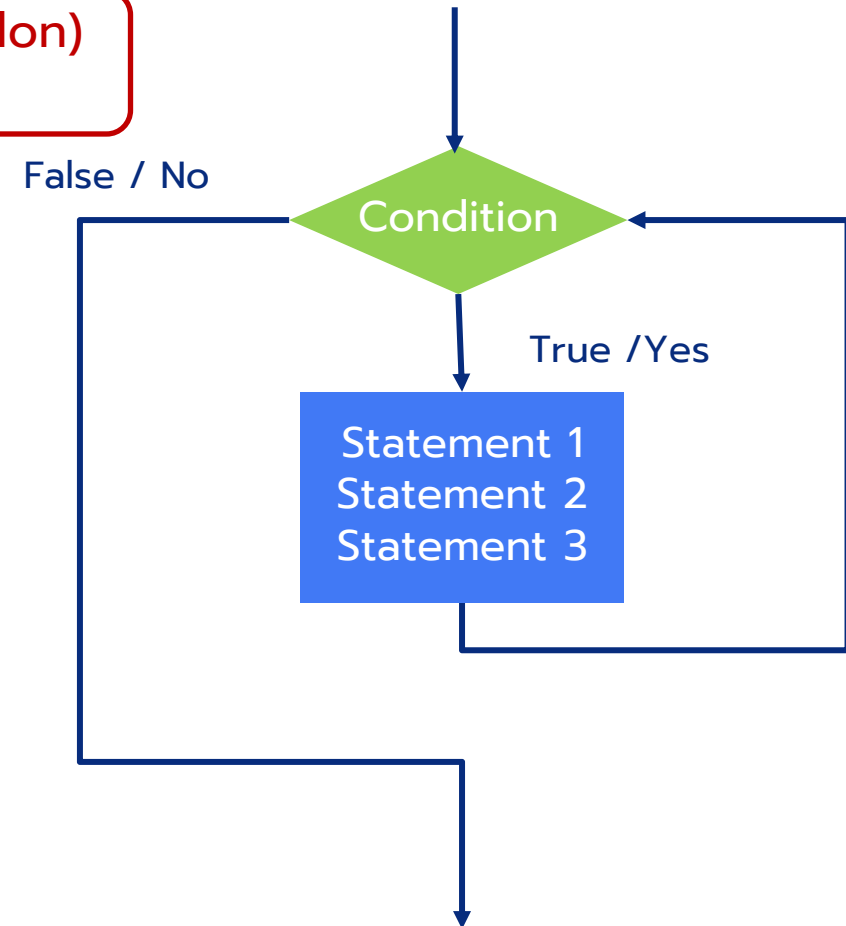
```
while condition:
    statement 1
    statement 2
    statement n
```

หลังเงื่อนไขจะจบด้วย : (colon) เสมอ

เงื่อนไขเป็นจริงเข้าทำ
โค้ดในส่วนนี้

วนไป
ตรวจสอบ
เงื่อนไข

การย่อหน้าใน Python ใช้การเคาะ
ช่องว่าง (space) 2 หรือ 4 ครั้ง



C
C
C
C
C
C
C
C
C
C
C

คำสั่งวนซ้ำ: while

- **Example 1: แสดงตัวเลขที่ตั้งแต่ 0 ถึง 5**

```
count = 0
while count < 5: #ใช้ตัวแปร count เป็นเงื่อนไข
    print(count)
    count += 1
```

print ก่อนเพิ่มค่า count

> Output

0
1
2
3
4

คำสั่งวนซ้ำ: while

- **Example 1: แสดงตัวเลขที่ตั้งแต่ 0 ถึง 5**

```
count = 0
while count <= 5: #ใช้ตัวแปร count เป็นเงื่อนไข
    print(count)
    count += 1
```

print ก่อนเพิ่มค่า count

```
count += 1
print(count)
```

ถ้าปรับโค้ดเพิ่มค่า count ก่อน print จะได้ผลลัพธ์อย่างไร ?

> Output

0
1
2
3
4
5

คำสั่งวนซ้ำ: while

- **Example 1: แสดงตัวเลขที่ตั้งแต่ 0 ถึง 5**

```
count = 0
while count <= 5: #ใช้ตัวแปร count เป็นเงื่อนไข
    print(count)
    count += 1
```

อยากแสดงตัวเลขจากมากไปน้อย จาก 5 ไป 0
ต้องแก้โค้ดจุดไหนบ้าง ?

> Output

5
4
3
2
1
0



Question 1

คำสั่งวนซ้ำ: while

- **Example 2: หาผลรวมตั้งแต่ 0 - 10**

```
num = 10
i = 1
while i <= num:
    total = total + i
    print('Round=', i, 'Cumulative Sum=', total)
    i + 1
```

```
print("Sum of first 10 number is:")
```

print ผลรวมนอก while

```
2 i = 1
3 while i <= num:
----> 4     total = total + i
5     print('Round=', i, 'Cumulative Sum=', total)
6     i + 1
```

NameError: name 'total' is not defined

คำสั่งวนซ้ำ: while

- **Example 2: หาผลรวมตั้งแต่ 0 - 10**

```
num = 10
i = 1
total = 0
while i <= num:
    total = total + i
    print('Round=', i, 'Cumulative Sum=', total)
    i + 1
print("Sum of first 10 number is:", total)
```

> Output

```
Round= 1 Cumulative Sum= 1
Round= 1 Cumulative Sum= 2
Round= 1 Cumulative Sum= 3
Round= 1 Cumulative Sum= 4
Round= 1 Cumulative Sum= 5
Round= 1 Cumulative Sum= 6
Round= 1 Cumulative Sum= 7
Round= 1 Cumulative Sum= 8
Round= 1 Cumulative Sum= 9
Round= 1 Cumulative Sum= 10
```

...

วนไม่รู้จบ

คำสั่งวนซ้ำ: while

- **Example 2: หาผลรวมตั้งแต่ 0 - 10**

```
num = 10
i = 1
total = 0
while i <= num:
    total = total + i
    print('Round=', i, 'Cumulative Sum=', total)
    i = i + 1

print("Sum of first 10 number is:", total)
```

```
i += 1
```

> Output

```
Round= 1 Cumulative Sum= 1
Round= 2 Cumulative Sum= 3
Round= 3 Cumulative Sum= 6
Round= 4 Cumulative Sum= 10
Round= 5 Cumulative Sum= 15
Round= 6 Cumulative Sum= 21
Round= 7 Cumulative Sum= 28
Round= 8 Cumulative Sum= 36
Round= 9 Cumulative Sum= 45
Round= 10 Cumulative Sum= 55
Sum of first 10 number is: 55
```

คำสั่งวนซ้ำ: while

- **Example 2: หาผลรวมตัวเลข 5 ตัว โดยให้ผู้ใช้งานป้อนผ่านแป้นพิมพ์**

```
num = 5
i = 1
total = 0
while i <= num:
    x = input('Enter your number %d :' %i)

    total = total + int(x)
    print('Round=', i, 'Cumulative Sum=', total)
    i = i + 1

print("Sum of first 10 number is:", total)
```

> Output

```
Enter your number 1 :10
Round= 1 Cumulative Sum= 10
Enter your number 2 :20
Round= 2 Cumulative Sum= 30
Enter your number 3 :30
Round= 3 Cumulative Sum= 60
Enter your number 4 :40
Round= 4 Cumulative Sum= 100
Enter your number 5 :50
Round= 5 Cumulative Sum= 150
Sum of first 10 number is: 150
```

คำสั่งวนซ้ำ: while

- **Example 3: ผลรวมของเลขคู่ตั้งแต่ 0 ถึง 10**

```
n = 10
total = 0
i = 1

while i <= n:
    if (i%2==0): # เป็นเลขคู่ถึงจะนำมารวม
        total = total + i
        i = i+1

print("The sum of even numbers is", total)
```



คำสั่งวนซ้ำ: while

- **Example 3: ผลรวมของเลขคู่ตั้งแต่ 0 ถึง 10**

```
n = 10
total = 0
i = 1

while i <= n:
    if (i%2==0): # ตรวจสอบว่าหาร 2 ลงตัวหรือไม่
        total = total + i
        i = i + 1
print("The sum of even numbers is", total)
```

> Output

The sum of even numbers is 30

2 + 4 + 6 + 8 + 10

คำสั่งวนซ้ำ: while

- ตัวอย่าง: แสดงเลขคู่/เลขคี่ จำนวน N ตัว

```
N = 5
i = 0; num = 0
while i < N:
    print(num)
    num += 2

    i += 1
```

> Output

0
2
4
6
8

```
N = 5
i = 0; num = 1
while i < N:
    print(num)
    num += 2

    i += 1
```

num ใช้ควบคุมการแสดงผล
ตัวเลขคู่/คี่

i ใช้นับจำนวนรอบ

> Output

1
3
5
7
9

คำสั่งวนซ้ำ: while

- ตัวอย่าง: จำลองการฝึกโมเดลจนกว่าค่าความผิดพลาดจะต่ำพอ

```
loss = 1.0
epoch = 0

while loss > 0.1:
    loss = loss * 0.7
    epoch += 1
    print(f"Epoch {epoch} | Loss: {loss:.4f}")

print("Training complete")
print(f"Total epochs: {epoch}")
```

จำลองการลด Loss แต่ละรอบ

Output

```
Epoch 1 | Loss: 0.7000
Epoch 2 | Loss: 0.4900
Epoch 3 | Loss: 0.3430
Epoch 4 | Loss: 0.2401
Epoch 5 | Loss: 0.1681
Epoch 6 | Loss: 0.1176
Epoch 7 | Loss: 0.0824
Training complete.
Total epochs: 7
```

คำสั่งวนซ้ำ: while

- ตัวอย่าง: นับจำนวนข้อมูลแต่ละคลาส (Class Distribution)

```
n = int(input("Enter number of samples: "))
cat = dog = bird = 0
count = 1

while count <= n:
    label = input(f"Label {count}: ")
    if label == "Cat":
        cat += 1
    elif label == "Dog":
        dog += 1
    elif label == "Bird":
        bird += 1
    count += 1
    print("\nClass Distribution")
    print("Cat :", cat)
    print("Dog :", dog)
    print("Bird :", bird)
```

>- Output

```
Enter number of samples: 8
Label 1: Cat
Label 2: Dog
Label 3: Cat
Label 4: Bird
Label 5: Dog
Label 6: Cat
Label 7: Dog
Label 8: Dog
```

ลองคิดต่อหากต้องการ
แสดงผลลัพธ์เป็น % ต้องแก้
โปรแกรมอย่างไร ?
Cat : 3 (37.50%)
Dog: 4 (50.00%)
Bird: 1 (12.50%)

```
Class Distribution
Cat : 3
Dog : 4
Bird : 1
```

คำสั่งวนซ้ำ: while

- ตัวอย่าง: โปรแกรมคำนวณ dot product จากข้อมูลที่รับทางแป้นพิมพ์

```
x_input = input("Enter vector x (comma-separated): ")
w_input = input("Enter vector w (comma-separated): ")

x_str = x_input.split(",")
w_str = w_input.split(",")
if len(x_str) != len(w_str):
    print("Error: Vectors must have the same length")
else:
    dot = 0
    i=0
    while i < len(x_str):
        xi = float(x_str[i])
        wi = float(w_str[i])
        dot += xi * wi
        i+=1

    print("Dot product:", round(dot, 2))
```

ตรวจสอบความยาว

คำนวณ dot product

> Output

```
Enter vector x (comma-separated): 2,3,5
Enter vector w (comma-separated): 0.5,0.2,0.1
Dot product: 2.1
```

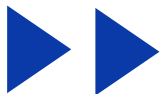
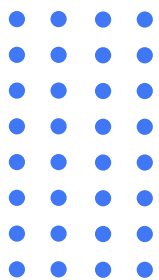


Question 2

ตอนไหนจะใช้ while

- **ทำงานลักษณะซ้ำๆ (Automate and repeat tasks):**
 - คำสั่ง while จะทำซ้ำโค้ดที่อยู่ในนั้นจนกว่าเงื่อนไขจะเป็นเท็จ
- **การทำงานอาจมีจำนวนรอบที่ไม่ชัดเจน (Indefinite Iteration):**
 - คำสั่ง while ในบางครั้งอาจจะไม่มีจำนวนรอบที่ไม่ชัดเจน เช่น เราต้องการรับตัวเลขจากผู้ใช้ไปเรื่อยๆ จนกว่าผู้ใช้จะอยากจบโปรแกรม **ไม่รู้ว่าคนใช้จะป้อนที่ตัว**
- **ทำงานซ้ำๆ ไม่รู้จบ (Infinite loop):**
 - คำสั่ง while จะมีลักษณะเป็น loop ไม่รู้จบได้ ถ้าเราไม่เขียนโปรแกรมเพื่อปรับเงื่อนไขให้เป็นเท็จ

คำสั่งวนซ้ำ for



ตอนไหนจะใช้ for

- มีการวนซ้ำที่แน่นอน (Definite Iteration):
 - หากค่าเฉลี่ยคะแนนของนักศึกษา 8 คน เราจะต้องวนซ้ำ 8 รอบ เพื่อรวมคะแนน



คำสั่ง for

- ตัวแปรลักษณะลำดับ (sequence) นิยมใช้คำสั่ง for วนซ้ำ
 - string 'Hello Python'
 - list [80, 85, 90, 75, 75, 65, 75, 70]
 - dictionary {'red': 1 , 'green': 2, 'bule':3}
 - set {0,1,2,3,4}
 - tuple (0,1,1,1,2,3,4)

คำสั่ง for

- คำสั่ง for

Syntax:

```
for element in sequence:
    statement 1
    statement 2
    statement n
```

จะจบด้วย : (colon) เสมอ

การย่อหน้าใน Python ใช้การเคาะ
ช่องว่าง (space) 2 หรือ 4 ครั้ง



True / Yes

Is this the last element
of this sequence?

False / No

Statement 1
Statement 2
Statement 3

คำสั่ง for

- **Example 1: แสดงตัวเลขใน List**

```
nums = [0, 2, 4, 6, 8, 10]

for n in nums:
    print(n)
```

> Output

```
0
2
4
6
8
10
```

คำสั่ง for

- **Example 2: หาค่าเฉลี่ยตัวเลขใน List**

```
numbers = [10, 20, 30, 40, 50]

total = 0
for i in numbers:
    total = total + i
list_size = len(numbers)
average = total / list_size

print('Average :', average)
```

> Output

Average : 30.0

คำสั่ง for

- **Example 3: หาผลรวมและค่าเฉลี่ยจำนวนมากกว่า 0**

```
total = 0
nums = [0, 1, 2, -1, -2, 3, 4, 5]
for i in nums:
    if i > 0:
        total += i

print('Total :', total)
print('Average :', total / len(nums))
```

>- Output

Total 15

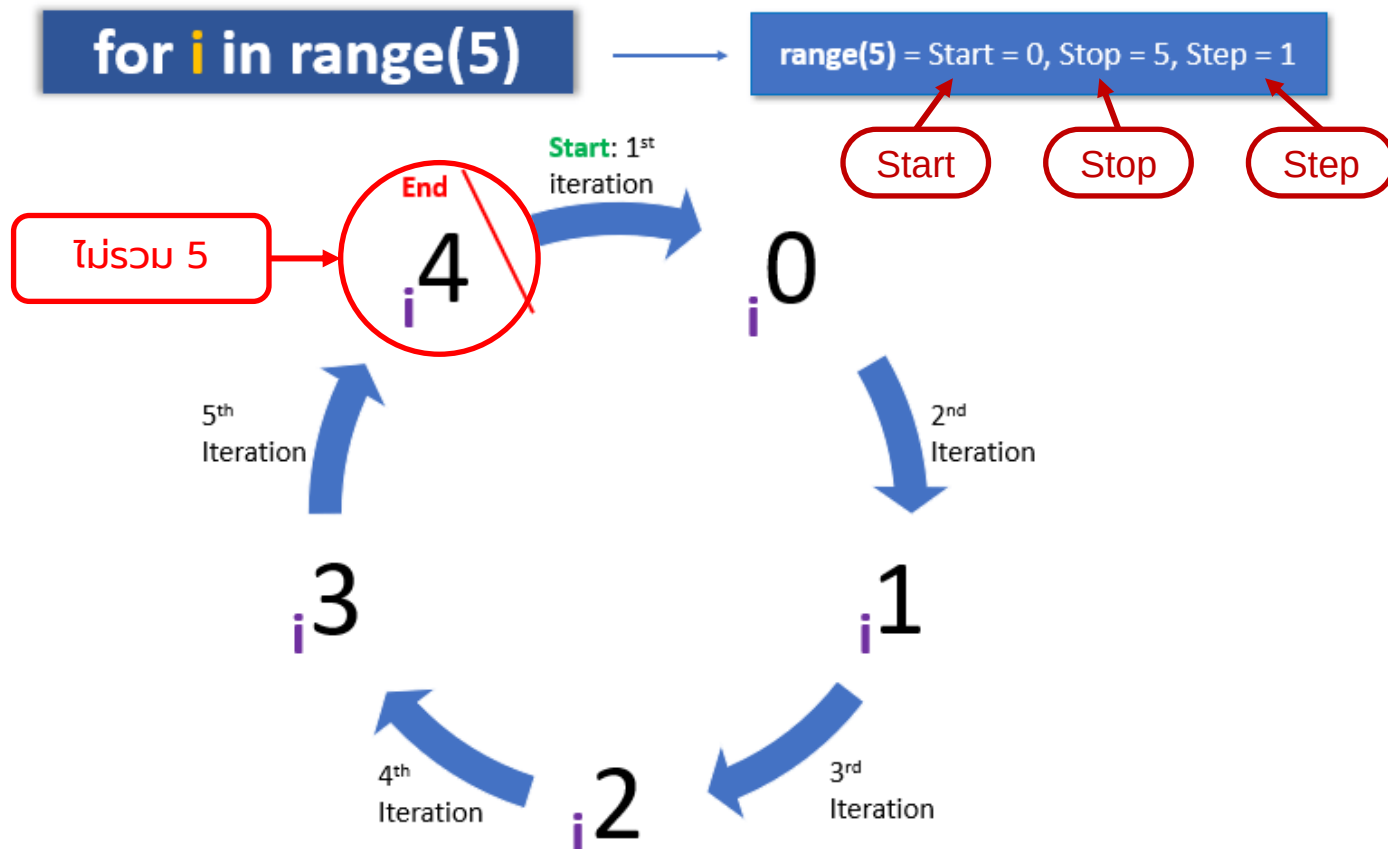
ลองทำดู

- ให้สร้าง list โดยเก็บตัวเลขจำนวนเต็มตั้งแต่ 1 ถึง 1000

```
nums = []  
i=1  
while i <= 1000:  
    nums.append(i)  
    i+=1  
  
print(nums)
```

การใช้คำสั่ง for ร่วมกับ range()

- range()** : สร้างชุดตัวเลข โดยกำหนดตัวเลขเริ่มต้น สิ้นสุด และการเพิ่มแต่ละรอบ



```
print(range(5))
```

```
range(0, 5)
```

```
print(list(range(5)))
```

```
[0, 1, 2, 3, 4]
```

ต้องแปลงเป็น list ก่อน
จึงจะ print ดูสมาชิกได้ภายในได้

การใช้คำสั่ง for ร่วมกับ range()

- **Example 3: การทำงานของฟังก์ชัน range()**

```
print(list(range(0)))  
print(range(1, 5))  
print(list(range(5)))  
print(list(range(1, 5)))  
print(list(range(1, 10, 2)))
```

↑ เพิ่มทีละ 2

>_ Output

```
[]  
  
range(1, 5)  
  
[0, 1, 2, 3, 4]  
  
[1, 2, 3, 4]  
  
[1, 3, 5, 7, 9]
```

การใช้คำสั่ง for ร่วมกับ range()

- **Example 3: แสดงตัวเลขตั้งแต่ 0-5**

```
for num in range(6):  
    print(num)
```

> Output

0
1
2
3
4
5

การใช้คำสั่ง for ร่วมกับ range()

- **Example 3: แสดงเลขคู่ตั้งแต่ 0-10**

```
for i in range (2, 11, 2):  
    print (i)
```

> Output

```
2  
4  
6  
8  
10
```

การใช้คำสั่ง for ร่วมกับ range()

- **Example 3: แสดงตัวเลขจากน้อยไปมาก 5-1**

```
for i in range (5, 0, -1):  
    print(i)
```

Step
เป็นลบได้

> Output

5
4
3
2
1

- หากรันโค้ดต่อไปนี้จะได้ผลลัพธ์อย่างไร

การใช้คำสั่ง for ร่วมกับ range()

- **Example 3: แสดงผลบวกสะสมตั้งแต่ 1 ถึง 10**

```
sum=0
for n in range(1,11):
    sum+=n
    print('Sum = ',sum)
```

> Output

```
sum = 1
sum = 3
sum = 6
sum = 10
sum = 15
sum = 21
sum = 28
sum = 36
sum = 45
sum = 55
```

การใช้คำสั่ง for ร่วมกับ range()

- **Example 4: แสดงเลขคู่เลขคี่ (for + range + if-else)**

```
for i in range(0, 6):  
    if i % 2 == 0:  
        print(i, 'is even number')  
    else:  
        print(i, 'is odd number')
```

> Output

```
Odd Number: 1  
Even Number: 2  
Odd Number: 3  
Even Number: 4  
Odd Number: 5
```

การใช้คำสั่ง for ร่วมกับ range()

- ตัวอย่าง: โปรแกรมคำนวณ Accuracy

```
actual = ["spam", "spam", "not spam", "spam", "not spam"]  
pred = ["spam", "not spam", "not spam", "spam", "spam"]
```

```
correct = 0  
for i in range(len(actual)):  
    print('index ', i, ': ', actual[i], ', ', pred[i], end=' : ' )  
  
    if actual[i] == pred[i]:  
        print("Correct")  
        correct += 1  
    else:  
        print("Wrong")  
  
print("Total correct =", correct)
```

> Output

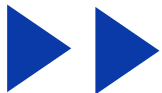
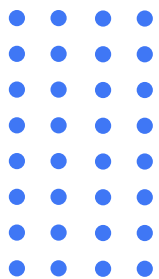
```
index 0 : spam , spam : Correct  
index 1 : spam , not spam : Wrong  
index 2 : not spam , not spam : Correct  
index 3 : spam , spam : Correct  
index 4 : not spam , spam : Wrong  
Total correct = 3
```



Question 4

- หากรันโค้ดต่อไปนี้จะได้ผลลัพธ์อย่างไร

คำสั่งโอนย้าย Transfer/Control statements



คำสั่งโอนย้าย: break

- เป็นคำสั่งที่ใช้สำหรับออกจากลูป หากพบคำสั่ง break จะทำการออกจากลูปทันที

```
while condition 1:  
    if (condition 2) :  
        Statement1  
        break  
    Statement 2  
Statement 3
```



คำสั่งโอนย้าย: break

- เป็นคำสั่งที่ใช้สำหรับออกจากลูป หากพบคำสั่ง break จะทำการออกจากลูปทันที

```
while เราจะรักกันตลอดไป :
    if (เขาไม่รัก 

```

→ เดี่ยวก็เจอคนใหม่

คำสั่งโอนย้าย: break

- **Example 1:** ตรวจสอบข้อมูลน้ำหนักที่มีค่าติดลบที่เก็บใน list

```
i=0
weights = [60,100,50,80,-20,70,75]
while i < len(weights):
    if weights[i] <=0: #ตรวจสอบจำนวนน้อยกว่า 0
        print('Found invalid value at index:',i,
              'value:',weights[i])
        break
    print(weights[i])
    i+=1
```

> Output

```
60
100
50
80
Found invalid value at index :
4 value: -20
```

คำสั่งโอนย้าย: break

- **Example 1: ตรวจสอบถ้าผู้ใช้ป้อน password จนกว่าจะถูกต้อง**

```
pw = 'Python1234'
while True:
    upass = input('Enter your password:')
    if upass==pw:
        print('Correct password')
        break
    else:
        print('Invalid password')
```

> Output

```
Enter your password:123456
Invalid password
Enter your password:python1234
Invalid password
Enter your password:Python1234
Correct password
```

คำสั่งโอนย้าย: break

- **Example: Coffee Shop**

> Output

```
==== Coffee Shop ====
```

```
1. Americano
```

```
2. Latte
```

```
3. Cappuccino
```

```
4. Exit
```

```
Choose menu: 1
```

```
You ordered Americano
```

```
==== Coffee Shop ====
```

```
1. Americano
```

```
2. Latte
```

```
3. Cappuccino
```

```
4. Exit
```

```
Choose menu: 4
```

```
Thank you
```

```
while True:
```

```
    print("\n==== Coffee Shop ====")
```

```
    print("1. Americano")
```

```
    print("2. Latte")
```

```
    print("3. Cappuccino")
```

```
    print("4. Exit")
```

```
    menu = input("Choose menu: ")
```

```
    if menu == "1":
```

```
        print("You ordered Americano")
```

```
    elif menu == "2":
```

```
        print("You ordered Latte")
```

```
    elif menu == "3":
```

```
        print("You ordered Cappuccino")
```

```
    elif menu == "4":
```

```
        print("Thank you")
```

```
        break
```

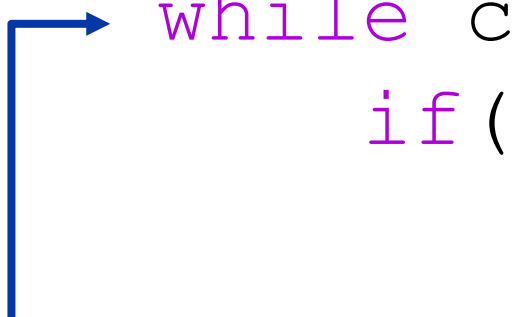
```
    else:
```

```
        print("Invalid menu")
```

คำสั่งโอนย้าย: continue

- **continue** เป็นคำสั่งที่ใช้สำหรับข้ามไปทำการวนซ้ำรอบต่อไป โดยข้ามส่วนของโค้ดที่อยู่หลังจากคำสั่ง continue

```
while condition 1:
    if (condition 2) :
        Statement 1
        continue
    Statement 2
    Statement 3
```

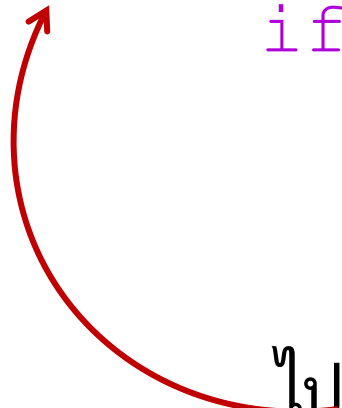


คำสั่งโอนย้าย: continue

- **Continue** : ข้ามส่วนของโค้ดแล้วเริ่มทำงานรอบต่อไป

```
while i < จำนวนคนคุย :
    if (คนปัจจุบัน i ยังไม่ถูกใจ) :
        คุยกับคนต่อไป
        i += 1
        continue
```

ไปกินข้าว
ไปดูหนัง
i += 1



เธอกินข้าวกับเราแล้วเธอไป
กินข้าวกับคนอื่นอีกไหม?

คำสั่งโอนย้าย: continue

- **Example 1:** หาผลรวมของจำนวนคี่ที่เก็บใน list

```
numbers = [5, 9, 10, 2, 3, 6, 8, 1, 7, 4]
i=0
print("Printing even numbers:")
while i < len(numbers):
    if numbers[i] % 2 == 1:
        i = i + 1
        continue
    print(numbers[i])
    i+=1
```

$i \Rightarrow 0$
 $\text{numbers}[i] \Rightarrow 1$
 $1 \% 2 \Rightarrow 1$

> Output

Printing even numbers:

10

2

6

8

4



Question 5

- หากรันโค้ดต่อไปนี้จะได้ผลลัพธ์อย่างไร

คำสั่ง pass

- **pass:** คำสั่งสำหรับหลีกเลี่ยงข้อผิดพลาดในกรณีที่ต้องการเขียนโค้ดเพิ่มเติมในภายหลัง

```
for number in range(6):
```

```
for number in range(6):  
    ^  
SyntaxError: incomplete input
```

```
for number in range(6):  
    pass
```

คำสั่ง pass

- pass ใช้กับคำสั่ง if และ while

```
nums = [1, 0, 2, 3, 0, 4, 5]
for i in nums:
    if (i <= 0):
        pass
        print('After pass')
    print(i)
```

คำสั่ง pass != continue

Output

```
1
After pass
0
2
3
After pass
0
4
5
```

การใช้คำสั่ง for ร่วมกับ break

- **Example 4:** ตรวจสอบข้อความกำกับรูปภาพเป็น 'dog' หรือ 'cat' เท่านั้น

```
labels = ('cat', 'cat', 'dog', 'fish')
for label in labels :
    if label not in ['cat', 'dog']:
        print('Invalid label:', label)
        break #ออกจากลูป
    print(label)
```

Output

```
cat
cat
dog
Invalid label: fish
```

การใช้คำสั่ง for ร่วมกับ break

- **Example 4: ค้นหาตัวเลขตัวเลขตามที่ใช้ป้อน ถ้าพบให้แสดงตำแหน่งที่พบ**

```
numbers = [1, 4, 7, 8, 15, 20, 35, 45, 55]
i = -1
unum = int(input('Enter your number:'))
for num in numbers:
    if num == unum :
        print('Found at index :', i)
        break
    i+=1
```

> Output

```
Enter your number:20
Found at index : 4
```

การใช้คำสั่ง for ร่วมกับ continue

- **Example 4: แสดงเฉพาะจำนวนที่มีค่ามากกว่า 0 (เขียนได้หลายแบบ)**

```
nums = [1, 0, 2, 3, 0, 4, 5]
for i in nums:
    if (i <= 0):
        continue
    print(i)
```

← ใช้ if-else ก็ได้

>- Output

1
2
3
4
5



Question 6

- หากรันโค้ดต่อไปนี้จะได้ผลลัพธ์อย่างไร





Question 7



จบสัปดาห์แรก

ตัวอย่างที่ซับซ้อน

- for ซ่อน for

```
person_info = [
    ["John", 70, 180],
    ["Emily", 0, 165],
    ["Michael", 85, 0],
    ["Sophia", 0, 0],
    ["Daniel", 78, 175]
]
```



ข้อมูลเป็น list ซ่อนใน list

ข้อมูลน้ำหนักและส่วนสูงที่มีความผิดปกติ
น้ำหนัก หรือ ส่วนสูงเป็น 0

```
cleaned_person_info = []
for person in person_info:
    if person[1]==0 or person[2]==0:
        continue
    cleaned_person_info.append(person)

for person in cleaned_person_info:
    print(person)
```

> Output

```
['John', 70, 180]
['Michael', 85, 190]
['Daniel', 78, 175]
```

ตัวอย่างที่ซับซ้อน

- จงเขียนโปรแกรมเพื่อกรองข้อมูลน้ำหนักและส่วนสูงที่มีความผิดปกติ

```
person_info = [
    ["John", 70, 180],
    ["Emily", 0, 165],
    ["Michael", 85, 0],
    ["Sophia", 0, 0],
    ["Daniel", 78, 175]
]
```



ข้อมูลเป็น list ซ้อนใน list

ข้อมูลน้ำหนักและส่วนสูงที่มีความผิดปกติ
น้ำหนัก หรือ ส่วนสูงเป็น 0

```
cleaned_person_info = []
for person in person_info:
    if person[1]==0 or person[2]==0:
        continue
    cleaned_person_info.append(person)

for person in cleaned_person_info:
    print(person)
```

> Output

```
['John', 70, 180]
['Michael', 85, 190]
['Daniel', 78, 175]
```

ใช้คำสั่ง for ทำงานกับ Tuple

```
numbers = (0, 1, 2, 3, 4, 5)
for number in numbers:
    print(number)
```

> Output

0
1
2
3
4
5

ใช้คำสั่ง for ทำงานกับ Dictionary

- **Example 1: แสดง key ของ dictionary**

```
days_dict = {  
    "Sun": "Sunday",  
    "Mon": "Monday",  
    "Tue": "Tuesday",  
    "Wed": "Wednesday",  
    "Thu": "Thursday",  
    "Fri": "Friday",  
    "Sat": "Saturday"  
}  
for key in days_dict:  
    print(key)
```

Output

```
Sun  
Mon  
Tue  
Wed  
Thu  
Fri  
Sat
```

ใช้คำสั่ง for ทำงานกับ Dictionary

- **Example 2: แสดงทั้ง key และ value ของ dict**

```
days_dict = {  
    "Sun": "Sunday",  
    "Mon": "Monday",  
    "Tue": "Tuesday",  
    "Wed": "Wednesday",  
    "Thu": "Thursday",  
    "Fri": "Friday",  
    "Sat": "Saturday"  
}  
  
for key in days_dict:  
    print(key, "->", days_dict[key])
```

Output

```
Sun -> Sunday  
Mon -> Monday  
Tue -> Tuesday  
Wed -> Wednesday  
Thu -> Thursday  
Fri -> Friday  
Sat -> Saturday
```

ฟังก์ชันอื่นๆ ที่นิยมใช้ร่วมกับคำสั่ง for

- **enumerate(), zip(), sorted()**

```

pets = ['Dog', 'Cat', 'Fish']
pet_th = ['สุนัข', 'แมว', 'ปลา']
print("\nenuemerated:\t")
#จะได้ index และ value
for i,p in enumerate(pets):
    print(i, "=", p)

print("\nzip'ed: ")
for en,th in zip(pets,pet_th):
    print(en, ":", th)

print("\nsorted:\t")
#จะได้ index และ value
for i,p in enumerate(sorted(pets)):
    print(i, "=", p)

```

> Output

enuemerated:

0 = Dog

1 = Cat

2 = Fish

zip'ed:

Dog : สุนัข

Cat : แมว

Fish : ปลา

enuemerated:

0 = Cat

1 = Dog

2 = Fish

ตัวอย่างการวนซ้ำ

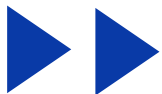
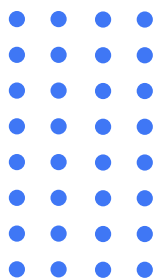
- โปรแกรมแสดงตัวเลขตั้งแต่เป็นแถวตามจำนวนที่กำหนด (c)

```
i=1
n=24
c=5
while i <= n:
    print("%02d" %i,end=' ')
    if i%c==0:
        print()
    i+=1
```

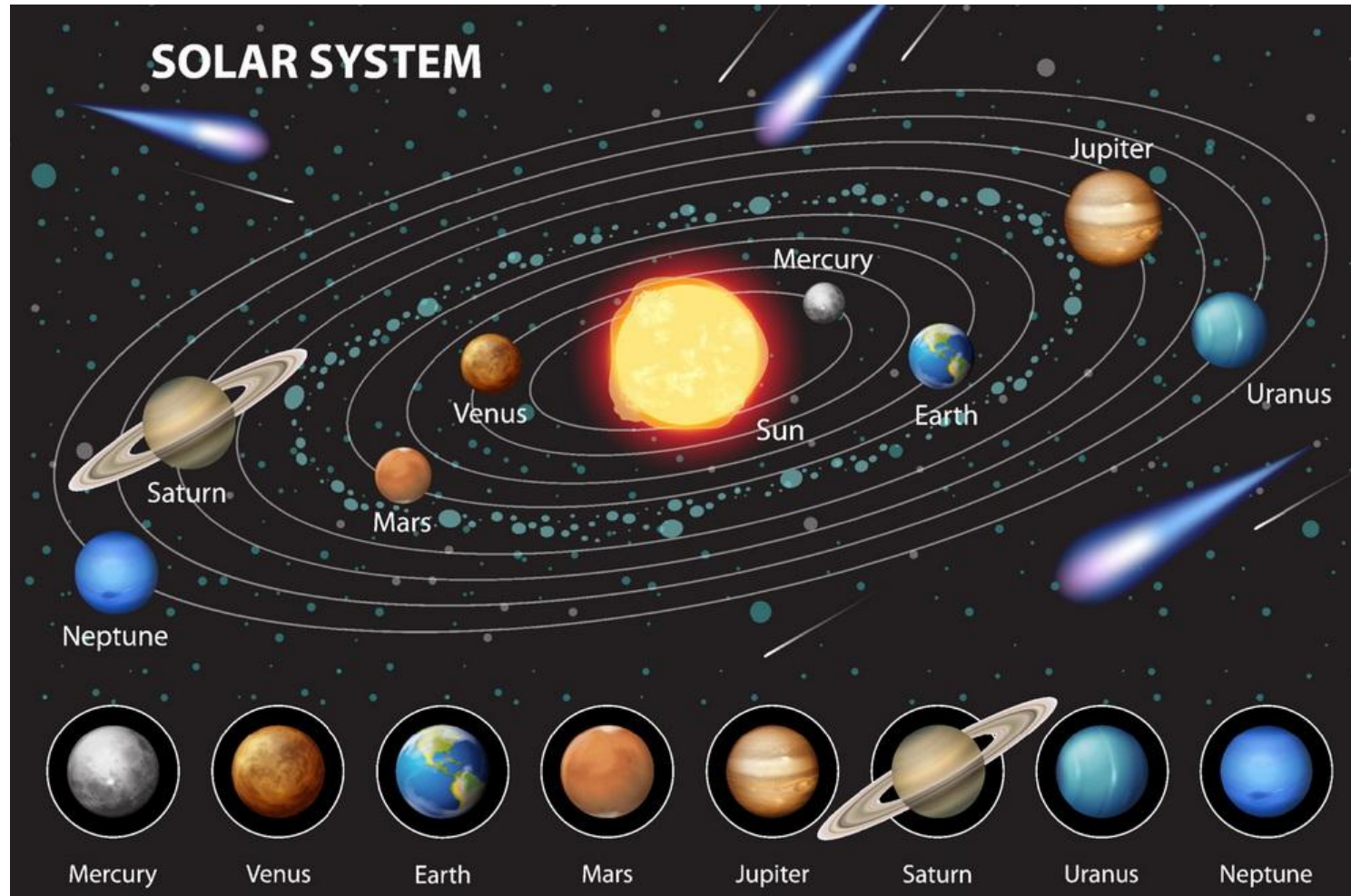
>_- Output

```
01 02 03 04 05
06 07 08 09 10
11 12 13 14 15
16 17 18 19 20
21 22 23 24
```

การวนซ้ำแบบซ้อนกัน (Nested loop)



ตัวอย่างการวนซ้ำซ้อนกันในชีวิตประจำวัน



การวนซ้ำแบบซ้อนกัน (Nested Loops)

- **Nested Loops** คือ **การใช้คำสั่งวนซ้ำซ้อนกัน**
- มีการวนซ้ำหนึ่งอยู่ภายในอีกการวนซ้ำหนึ่ง
 - ตัวอย่างเช่น while ซ้อน while หรือ for ซ้อน for
 - สามารถซ้อนกันได้มากกว่า 2 ระดับ

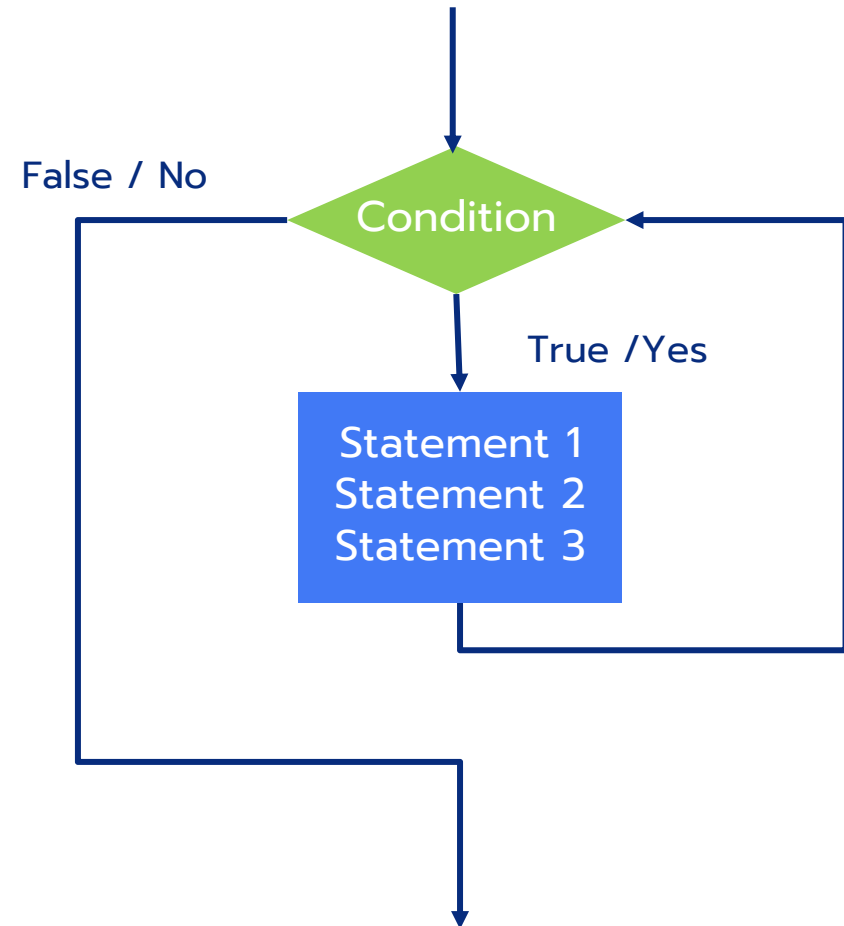
คำสั่ง while

- คำสั่ง while

Syntax:

```
while condition :  
    statement 1  
    statement 2  
    statement n
```

การย่อหน้าใน Python ใช้การเคาะ
ช่องว่าง (space) 2 หรือ 4 ครั้ง



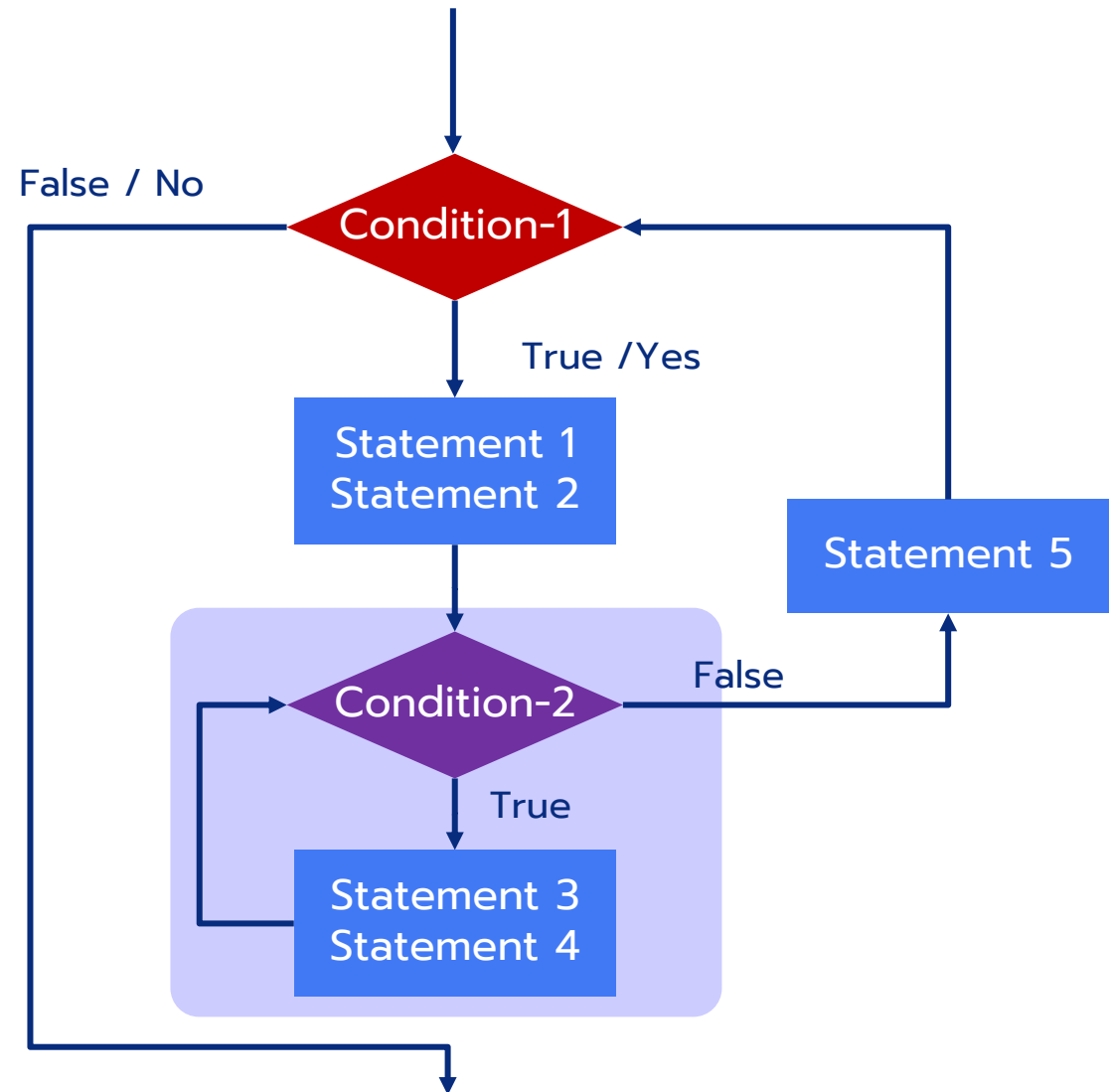
คำสั่ง while ซ้อน while

- คำสั่ง nested while

Syntax:

```
while condition-1 :
    statement 1
    statement 2
    while condition-2 :
        statement 3
        statement 4
        statement 5
```

การย่อหน้า



ตัวอย่างการวนซ้ำแบบซ้อนกัน

- โปรแกรมแสดง * เท่ากับจำนวน row และ col

```
col = 3
row = 4
i=0
j=0
```



>- Output

```
* * *
* * *
* * *
* * *
} row = 4
{ col = 3
```

ตัวอย่างการวนซ้ำแบบซ้อนกัน

- โปรแกรมแสดง * เท่ากับจำนวน row และ col

```
col = 3
row = 4
i=0
j=0
while i < row:
    while j < col:
        print('*', end='')
```

> Output

*
*
*
*

↓
Infinity loop

ตัวอย่างการวนซ้ำแบบซ้อนกัน

- โปรแกรมแสดง * เท่ากับจำนวน row และ col

```
col = 3
row = 4
i=0
j=0
while i < row:
    while j < col:
        print('*', end=' ')
        j+=1
    i+=1
    print()
```

>- Output

```
*
*
*
```

ได้ * มา 3 อัน แะะ

ตัวอย่างการวนซ้ำแบบซ้อนกัน

- โปรแกรมแสดง * เท่ากับจำนวน row และ col

```
col = 3
row = 4
i=0
j=0
while i < row:
    j=0
    while j < col:
        print('*', end=' ')
        j+=1
    i+=1
    print()
```

> Output

```
*
*
*
*
*
*
*
*
*
*
*
*
```

ได้ * มา 12 อัน

ต้องการแสดงบรรทัด
ละ 4 อัน

ตัวอย่างการวนซ้ำแบบซ้อนกัน

- โปรแกรมแสดง * เท่ากับจำนวน row และ col

```
col = 3
row = 4
i=0
j=0
while i < row:
    j=0
    while j < col:
        print('*',end=' ')
        j+=1
    i+=1
```

> Output

ได้ * มา 12 อัน
คราวนี้แน่นอน



ตัวอย่างการวนซ้ำแบบซ้อนกัน

- โปรแกรมแสดง * เท่ากับจำนวน row และ col

```
col = 3
row = 4
i=0
j=0
while i < row:
    j=0
    while j < col:
        print('*',end=' ')
        j+=1
    print()
    i+=1
```

> Output

```
* * *
* * *
* * *
* * *
```



ตัวอย่างการวนซ้ำแบบซ้อนกัน

- โปรแกรมแสดงตัวเลขลำดับแถว เท่ากับจำนวน คอลัมน์

```
col = 4
row = 3
i=0
j=0
while i < row:
    j=0
    while j < col:
        print(i,end=' ')
        j+=1
    print()
    i+=1
```

> Output

```
0 0 0 0
1 1 1 1
2 2 2 2
```

Question 8

- หากรันโค้ดต่อไปนี้จะได้ผลลัพธ์ อย่างไร

ตัวอย่างการวนซ้ำแบบซ้อนกัน

- โปรแกรมแสดง * เป็นสามเหลี่ยม

```
row = 4
i=0
```



> Output

```
*
* *
* * *
* * * * } row = 4
```

ตัวอย่างการวนซ้ำแบบซ้อนกัน

- โปรแกรมแสดง * เป็นสามเหลี่ยม

```
row = 4
i=0
j=0
while i < row:
    j=0
    while j <= i:
        print('*',end=' ')
        j+=1
    print()
    i+=1
```

> Output

```
*
* *
* * *
* * * *
```





Question 9

- หากรันโค้ดต่อไปนี้จะได้ผลลัพธ์อย่างไร



ตัวอย่างการวนซ้ำแบบซ้อนกัน

- โปรแกรมแสดงแถวและหลักของเมทริกซ์ (Matrix)

```
for i in range(____, ____):
    # nested loop
    for j in range(____, ____):
        _____
    print()
```

> Output

```
(1,1) (1,2) (1,3) (1,4)
(2,1) (2,2) (2,3) (2,4)
(3,1) (3,2) (3,3) (3,4)
(4,1) (4,2) (4,3) (4,4)
```



ตัวอย่างการวนซ้ำแบบซ้อนกัน

- โปรแกรมแสดงแถวและหลักของเมทริกซ์ (Matrix)

```
for i in range(1, 5):
    # nested loop
    for j in range(1, 5):
        print('(', i, ', ', j, ') ', end=' ', sep='')
    print()
```

> Output

```
(1,1) (1,2) (1,3) (1,4)
(2,1) (2,2) (2,3) (2,4)
(3,1) (3,2) (3,3) (3,4)
(4,1) (4,2) (4,3) (4,4)
```



ตัวอย่างการวนซ้ำแบบซ้อนกัน

- โปรแกรมสร้างข้อความจาก list ของคำ

```
name = ['Many', 'Jany']  
lang = ['Python', 'Java', 'You']  
for i in name:  
    for j in lang:  
        print(i, ' love' ,j)
```

> Output

```
Many love Python  
Many love Java  
Many love You  
Jany love Python  
Jany love Java  
Jany love You
```

ตัวอย่างการวนซ้ำแบบซ้อนกัน

- โปรแกรมแสดงตัวเลขเป็นสามเหลี่ยม

```
for i in range(1, 6):
    for j in range(1, ____):
        _____
    _____
```

> Output

```
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
```



ตัวอย่างการวนซ้ำแบบซ้อนกัน

- โปรแกรมแสดงตัวเลขเป็นสามเหลี่ยม

```
for i in range(1, 6):
    for j in range(1, i+1):
        print(i, end=" ")
    print("\n")
```

> Output

```
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
```



Question 10

- หากรันโค้ดต่อไปนี้จะได้ผลลัพธ์อย่างไร

คำสั่ง for บรรทัดเดียว

- การแปลงสตริงเป็นลิสต์ตัวเลขด้วย for บรรทัดเดียว

```
nums = input('Enter numbers:').split(',')  
for i in range(len(nums)):  
    nums[i] = int(nums[i])  
print(nums)
```

> Output

```
Enter numbers:10,20,30,40,50  
[10, 20, 30, 40, 50]
```

```
nums = input('Enter numbers:').split(',')  
nums = [int(n) for n in nums]  
print(nums)
```

> Output

```
Enter numbers:10,20,30,40,50  
[10, 20, 30, 40, 50]
```

คำสั่ง for บรรทัดเดียว

- คำสั่ง for สามารถเขียนบรรทัดเดียวได้

```
odd = [1, 5, 7, 9]
even = [i + 1 for i in odd]
print(even)
```

แปลงเป็น list ของเลขคู่

> Output

[2, 6, 8, 10]

เอาช่องว่างออก

```
text = ["Hello ", "World ", "Hi"]
new_list = [w.strip() for w in text]
print(new_list)
```

> Output

['Hello', 'World', 'Hi']

การวนซ้ำแบบซ้อนกันร่วมกับ list

- ต้องการหาผลรวมของสมาชิกทั้งหมดภายใน list ที่ซ้อนใน list

```
list_of_lists = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

หาผลรวมของสมาชิกทั้งหมด

Output

45



```
sum(list_of_lists)
```

```
-----
TypeError                                Traceback (most recent call last)
<ipython-input-9-ecc93ce2ff71> in <cell line: 1>()
----> 1 sum(list_of_lists)
```

TypeError: unsupported operand type(s) for +: 'int' and 'list'

การวนซ้ำแบบซ้อนกันร่วมกับ list

- โปรแกรมคำนวณผลรวมสมาชิกในเมตริกซ์

```
matrix = [[1, 2], [3, 4], [5, 6]]
total = 0

for row in matrix:
    for col in row:
        total += col
        print('>> current sum (col):', total)
    print('current sum (row):', total)

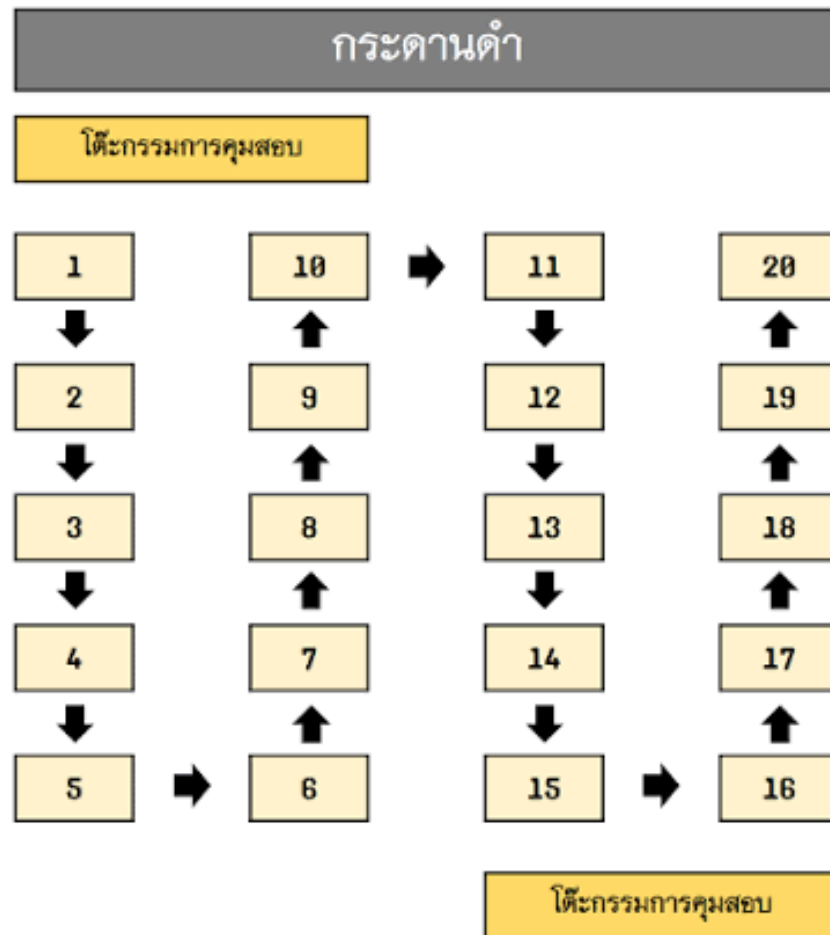
print("total sum:", total)
```

>> Output

```
>> current sum (col): 1
>> current sum (col): 3
current sum (row): 3
>> current sum (col): 6
>> current sum (col): 10
current sum (row): 10
>> current sum (col): 15
>> current sum (col): 21
current sum (row): 21
```

ตัวอย่างการวนซ้ำ: โปรแกรมจัดผังเลขที่นั่งสอบ

- แนวคิด: จะแสดงผลตรงๆ ไม่ได้ ต้องเก็บค่าไว้ก่อน



ตัวอย่างการวนซ้ำ: โปรแกรมจัดผังเลขที่นั่งสอบ

```

seat_nums = 16
row = 5

seats = [[0]*row] # [[0, 0, 0, 0, 0]]
cur_row = cur_col = 0

for i in range(1, seat_nums + 1):
    seats[cur_row][cur_col] = i
    cur_col += 1

    if i % row == 0 and i != seat_nums:
        seats.append([0]*row)
        cur_row += 1
        cur_col = 0

# ทดสอบแสดงผลแบบ Row
print("แสดงผลตาม Row")
for r in seats:
    print(r)

```

> Output

```

[[0, 0, 0, 0, 0]]
แสดงผลตาม Row
[1, 2, 3, 4, 5]
[6, 7, 8, 9, 10]
[11, 12, 13, 14, 15]
[16, 0, 0, 0, 0]

```

ตัวอย่างการวนซ้ำ: โปรแกรมจัดผังเลขที่นั่งสอบ

```
print("\nแสดงผลตาม Column")
for i in range(row):
    for j in range(len(seats)):
        if i < len(seats[j]):
            print(f'({j},{i}) {seats[j][i]:02d}', end=' ')
    print()
```

แสดงผลตาม Row

[1, 2, 3, 4, 5]

[6, 7, 8, 9, 10]

[11, 12, 13, 14, 15]

[16, 0, 0, 0, 0]

> Output

แสดงผลตาม Column

```
(0,0) 01 (1,0) 06 (2,0) 11 (3,0) 16
(0,1) 02 (1,1) 07 (2,1) 12 (3,1) 00
(0,2) 03 (1,2) 08 (2,2) 13 (3,2) 00
(0,3) 04 (1,3) 09 (2,3) 14 (3,3) 00
(0,4) 05 (1,4) 10 (2,4) 15 (3,4) 00
```

ตัวอย่างการวนซ้ำ: โปรแกรมจัดผังเลขที่นั่งสอบ

```
for i in range(row):
    for j in range(len(seats)):
        if i < len(seats[j]):
            new_i = i
            if j%2==1:
                new_i = row-i-1
            print(f"({j},{i},{new_i}) ",end='')
            if new_i< len(seats[j]):
                print('%02d' %seats[j][new_i], end=' ')
            print()
```

> Output

```
(0,0,0) 01 (1,0,4) 10 (2,0,0) 11 (3,0,4) 00
(0,1,1) 02 (1,1,3) 09 (2,1,1) 12 (3,1,3) 00
(0,2,2) 03 (1,2,2) 08 (2,2,2) 13 (3,2,2) 00
(0,3,3) 04 (1,3,1) 07 (2,3,3) 14 (3,3,1) 00
(0,4,4) 05 (1,4,0) 06 (2,4,4) 15 (3,4,0) 16
```

Question 11

- หากรันโค้ดต่อไปนี้จะได้ผลลัพธ์อย่างไร



Question 12

- หากรันโค้ดต่อไปนี้จะได้ผลลัพธ์อย่างไร



Question 13

- หากรันโค้ดต่อไปนี้จะได้ผลลัพธ์อย่างไร





Question 14

- หากรันโค้ดต่อไปนี้จะได้ผลลัพธ์อย่างไร

การวนซ้ำแบบซ้อนกันร่วมกับ list

- **คำนวณผลบวกของเมตริกซ์**

```
A = [[1, 2],  
      [3, 4]]  
B = [[5, 6],  
      [7, 8]]  
result = []  
for i in range(len(A)):  
    row = []  
    for j in range(len(A[0])):  
        row.append(A[i][j] + B[i][j])  
    result.append(row)  
  
print("Matrix Addition Result:")  
for row in result:  
    print(row)
```

> Output

```
Matrix Addition Result:  
[6, 8]  
[10, 12]
```

การวนซ้ำแบบซ้อนกันร่วมกับ list

- โปรแกรมคำนวณผลคูณเมทริกซ์

Example : $A = \begin{bmatrix} 0 & -1 \\ 0 & 1 \end{bmatrix}_{2 \times 2}$ และ $B = \begin{bmatrix} 1 & 0 & 2 \\ 3 & -1 & 0 \end{bmatrix}_{2 \times 3}$

$$AB = \begin{bmatrix} 0 & -1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 2 \\ 3 & -1 & 0 \end{bmatrix}$$

$$= \begin{bmatrix} (0 \times 1) + (-1 \times 3) & (0 \times 0) + (-1 \times -1) & (0 \times 2) + (-1 \times 0) \\ (0 \times 1) + (1 \times 3) & (0 \times 0) + (1 \times -1) & (0 \times 2) + (1 \times 0) \end{bmatrix}$$

$$= \begin{bmatrix} -3 & 1 & 0 \\ 3 & -1 & 0 \end{bmatrix}_{2 \times 3}$$

การวนซ้ำแบบซ้อนกัน

- โปรแกรมคำนวณผลคูณเมทริกซ์

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}, B = \begin{bmatrix} 7 & 8 \\ 9 & 10 \end{bmatrix}$$

```
for i in range(len(A)):          # rows of A (3)
    row = []
    for j in range(len(B[0])):   # columns of B (2)
        sum = 0
        for k in range(len(B)): # หาผลรวม
            sum += A[i][k] * B[k][j]
        row.append(sum)
    result.append(row)

print("Matrix Multiplication Result:")
for row in result:
    print(row)
```

Output

Matrix Multiplication Result:
[25, 28]
[57, 64]
[89, 100]

Question 15

- หากรันโค้ดต่อไปนี้จะได้ผลลัพธ์อย่างไร



บทวนท้ายบท

- คำสั่ง **while**
 - วงซ้ำตามเงื่อนไข จำนวนรอบไม่แน่นอน ต้องระวังการวนซ้ำไม่สิ้นสุด
- คำสั่ง **for**
 - วงซ้ำตามลำดับ มีจำนวนรอบแน่นอน ใช้กับข้อมูลลำดับ เช่น list tuple
- คำสั่ง **ไอนย้าย**
 - break: ออกจากการวนซ้ำทันที
 - continue: ข้ามคำสั่งที่เหลือในรอบปัจจุบัน ใช้เมื่อต้องการเลือกข้ามบางกรณี
- **Nested Loop**
 - การวนซ้ำซ้อนกันหลาย